

[Home](#) › [My Courses](#) › [Risk Management](#) › [M3: Estimating Tail Loss with Deep Learning](#) › Lesson Notes

Lesson Notes

MODULE 3 | LESSON 3

FORECASTING VOLATILITY WITH TRANSFORMERS

Reading Time	25 minutes
Prior Knowledge	GARCH, Value at Risk (VaR)
Keywords	Transformer architecture, Attention mechanism, Encoder-Decoder model

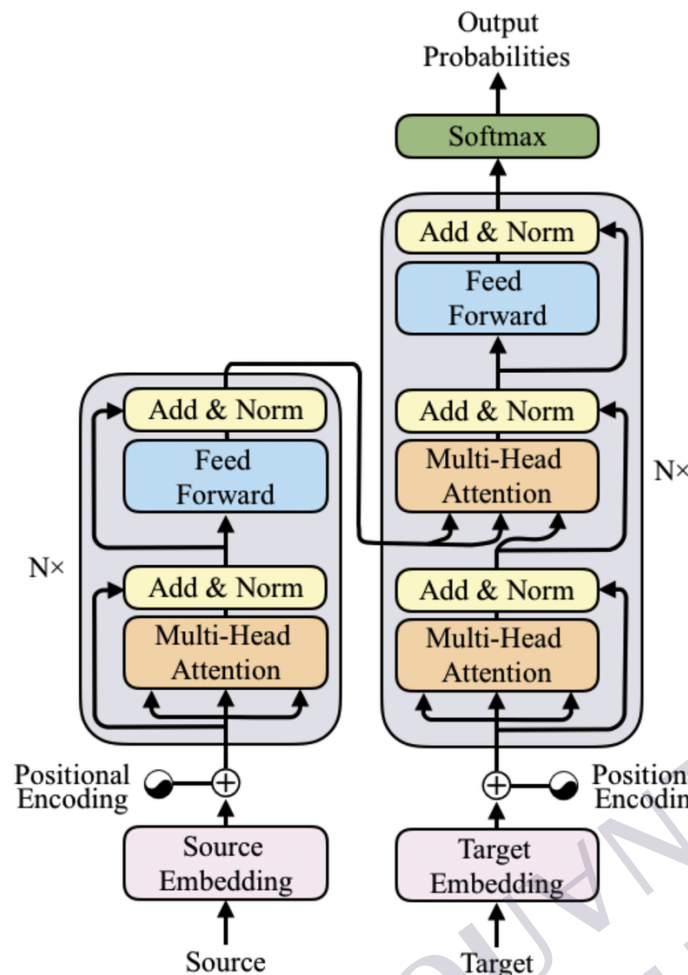
In the last two lessons, we started applying deep learning to risk management challenges, namely estimating Value at Risk. In this lesson and the next, we review an approach that takes advantage of even more recent neural network research.

1. Introducing Transformers

In the following, we attempt a high-level overview of the most important concepts related to transformers, probably the most important deep learning technology discovery in the last several years—and a technology whose potential in various domains continues to surprise and delight researchers. The caveats from the beginning of lesson one apply no less here: In this Risk Management course, we will focus on the implementation of this state-of-the-art technology toward risk management ends. In order to examine and manipulate the Python code implementation at our disposal (our focus in the next lesson), we need to have a big picture understanding that includes the new and important computational concepts related to this new class of neural network model. That is our goal for this lesson.

Thorough and detailed investigations (of varying degrees of complexity and difficulty) into the theory that supports our application is absolutely available, much of it free and on the internet. With that said, in this lesson we bring together what we already know about deep learning and supplement it with the most recent research. Our goal is to make use of these risk management techniques that appear to outperform techniques which, only a few years ago, seemed cutting edge.

Figure 1: The Transformer and Its Components

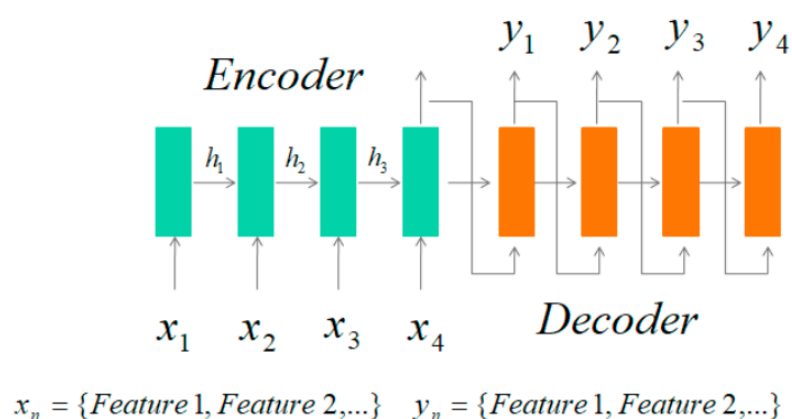


Source: Zhang, Zheng et al. "BLISS: Robust Sequence-to-Sequence Learning via Self-Supervised Input Representation." *Journal of LaTeX Class Files*, vol. 14, no. 8, 2021, pp. 1–9, <https://arxiv.org/abs/2204.07837>.

2. The Encoder-Decoder Model

Let's start with the basics with which you may already be familiar. As you can see from the left-most diagram in the visualization above, you can think of the transformer model as a class of encoder-decoder model. The major difference is that encoder-decoder models for sequence-to-sequence tasks (such as time-series prediction) usually employ recurrent neural networks (RNNs), like gated recurrent units (GRUs) or long-short term memory (LSTM) units. In this case, the transformer uses the attention mechanism instead. We will discuss attention below, but first, we will provide a review of the roles of the encoder and decoder, in the general case and then in the case of our transformer model.

Figure 2: Basic Encoder-Decoder Model for Time-Series Prediction



Source: Hwang, Soonsang, et al. "A Novel Time Series based Seq2Seq Model for Temperature Prediction in Firing Furnace Process." *Procedia Computer Science*, vol. 155, 2019, p. 20, <https://www.sciencedirect.com/science/article/pii/S1877050919309196>.

In general, the following is true about the encoder and decoder in an encoder-decoder model:

- The encoder is itself a neural network: The encoder performs feature extraction; that is, it projects the inputs into a feature space, mapping each input to a feature vector (also called a latent vector). This vector is meant to incorporate all the important information about that input and its relationship to all other inputs (thanks to the attention mechanism to be covered below) (Asadi 137). The feature space into which the inputs are projected needs to be learned, similar to any other neural network parameterization.
- The decoder, also a neural network, generates
- the output, in our case a prediction. It takes the feature vector from the encoder as its input and maps each vector to the output space (Asadi 137).

The decoder consists of the same number of layers as the encoder, each of which includes these same sublayers but in addition to these (the MHSA and the FFN) lies a third sublayer, the "encoder-decoder attention" sublayer, providing the crucial interface between the encoder and the decoder stacks.

Another difference between the encoder and the decoder is that the decoder takes as its input the output from the previous decoder layer. This has the desired result that the decoder cumulatively incorporates all the previous observations in the sequence of data.

Now in the case of the transformer, this is still true but some details of the architecture change: The cells are no longer recurrent like GRUs and LSTMs. Instead, we say that the encoder consists of a number of transformer layers, each of which consists of two mechanisms or sublayers:

1. multi-head self-attention (MHSA) mechanism
2. fully connected feed-forward network (FFN)

Also, and this is crucial, in the time-series prediction application of the transformer, which we use in this and the next lesson, there is no need for the decoder. Instead, we compare the output of the encoder to the next time-step's observations and train by reducing the error between these two. The YouTube video, which is required reading or viewing, provides a good overview of the concepts and context; on the other hand, it does not speak specifically to the time-series application and therefore describes in detail the relationship between the encoder and decoder. The transformer model is outperforming most other models by far, especially for higher dimensional data sets, so it is worth your time to grasp as much of this as possible. However, since this is a Risk Management course and not a Machine Learning course, the quizzes will not include questions specific to the video material, though the video will likely help you comprehend the rest of the transformer technology that is quizzed.

3. Attention

Perhaps the most intuitive way to understand attention is by analogy. Following Poupart, let us consider a database consisting of **values**. Each value is associated with a **key**, by which we refer to the value. Even better, we can say that we call only recall or select a value by reference to its corresponding key. Now, say we need some information from the database—this is our **query**. We

assume that the answer to the question is not straightforward: There are multiple possible answers, some more likely or appropriate than the others. Alternatively, in the language of the attention mechanism, we could say that the query is complex such that there is not a trivial linear mapping between the query and a single particular key/value pair. Rather, we can assign probabilities to the keys for each query. (Where do the probabilities come from? They have to be learned, via extensive training.)

Vaswani et al. in their paper "Attention is All You Need" introduced transformers and their immense power over the world. The authors define this pivotal concept of attention as follows: "An attention function can be described as mapping a query and a set of key-value pairs to an output, where the query, keys, values, and output are all vectors. The output is computed as a weighted sum of the values, where the weight assigned to each value is computed by a compatibility function of the query with the corresponding key" (Vaswani et al. 3).

Vaswani et al. define the attention scoring function as returning a weighted attention (where the weights equate to a probability distribution). Any given query is related to multiple keys according to a similarity ("compatibility"), which is expressed as a probability distribution. The key in turn corresponds directly (deterministically, not probabilistically) to a value. The probability-weighted combination of the values is our output or prediction. We will talk more about this similarity calculation in the next lesson; in the meantime, this preview of the concepts should help you appreciate more deeply the introduction to the transformer architecture provided by Ramos-Perez et al.

4. Conclusion

Now that we have reviewed the major concepts of attention and transformer architecture in this lesson, we will go into more details and look at the code that implements this cutting edge solution.

References

- *CodeEmporium*. "Transformer Neural Networks - EXPLAINED! (Attention is all you need)." *YouTube*, <https://www.youtube.com/watch?v=TQQIZhbc5ps>.
- Hwang, Soonsang, et al. "A Novel Time Series based Seq2Seq Model for Temperature Prediction in Firing Furnace Process." *Procedia Computer Science*, vol. 155, 2019, p. 20, <https://www.sciencedirect.com/science/article/pii/S1877050919309196>.
- Poupart, Pascal. "CS480/680 Lecture 19: Attention and Transformer Networks." *YouTube*, 16 July 2019, https://www.youtube.com/watch?v=OyFJWRnt_AY.
- Ramos-Perez, Eduardo, et al. "Multi-Transformer: A New Neural Network-Based Architecture for Forecasting S&P Volatility." *Mathematics*, vol. 9, no. 15. <https://www.mdpi.com/2227-7390/9/15/1794>.
- Vaswani, Ashish, et al. "Attention is All You Need." *arXiv*, 2017. <https://arxiv.org/abs/1706.03762?context=cs>
- Zhang, Zheng, et al. "BLISS: Robust Sequence-to-Sequence Learning via Self-Supervised Input Representation." *Journal of LaTeX Class Files*, vol. 14, no. 8, 2021, pp. 1–9, <https://arxiv.org/abs/2204.07837>.

